

L'ALGORITHME DE BOYER-MOORE-HORSPOOL

Un algorithme de recherche de chaînes de caractères



INTRODUCTION À LA RECHERCHE DE MOTIFS

Algorithme naïf de recherche

ALGORITHME NAÏF DE RECHERCHE

- **Objectif de l'algorithme :**

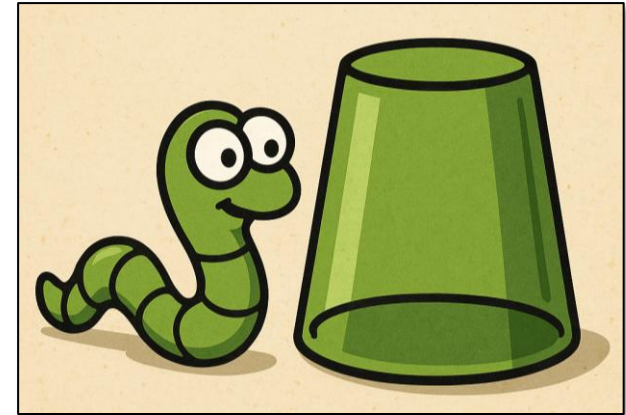
- Trouver la ou les occurrences d'un **motif** (mot isolé; extrait d'une phrase; suite de nucléotides pour la recherche dans une séquence d'ADN, etc.).
- La fonction de recherche prendra en argument :
 - le texte source
 - le motif à chercher.
- La fonction retournera suivant nos besoins :
 - Un booléen attestant de la présence ou de l'absence du motif dans le texte.
 - Le nombre d'occurrences du motif dans le texte.
 - L'index de la 1^{ère} occurrence du motif dans le texte.
 - Une liste contenant les index des occurrences du motif dans le texte.

ALGORITHME NAÏF DE RECHERCHE

- **Description du processus de recherche naïve :**

- Pour illustrer et tester le fonctionnement de l'algorithme nous travaillerons avec le texte suivant :

- *Un ver vert va vers un verre vert à l'envers.*



- Nous pourrions par exemple vérifier la présence du mot « **vert** » dans ce texte.

ALGORITHME NAÏF DE RECHERCHE

- **Description du processus de recherche naïve :**
 - Pour vérifier la présence du **motif**, nous utiliserons une **fenêtre** de vérification qui se déplacera de gauche vers la droite.
 - Cette fenêtre aura pour taille, la **taille** du **motif** à chercher dans le **texte**.

ALGORITHME NAÏF DE RECHERCHE

- **Description du processus de recherche naïve :**

- Dans la recherche naïve, la fenêtre de recherche est déplacée vers la droite par incrément de 1 pour le pas.
- Nous allons donc devoir définir la plage de déplacement de cette fenêtre, qui sera de **$i = 0$** pour le départ à **$i = \text{len}(\text{texte}) - \text{len}(\text{motif})$** pour la dernière position.
- L'image suivante illustre le fonctionnement décrit précédemment :

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44		
texte :	U	n		v	e	r		v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à		l	'	e	n	v	e	r	s	.		
motif :	v	e	r	t																																								v	e	r	t

↑

i_dep = 0

--> sens de déplacement de la fenêtre -->

↑

i_fin = 45 - 4 = 41

ALGORITHME NAÏF DE RECHERCHE

- Travail à faire :

- Ouvrir **Thonny** et créer un nouveau fichier que vous nommerez :
 - Recherche `naive.py`
- Déclarer la variable `le_ver` avec le texte de test :
 - *Un ver vert va vers un verre vert à l'envers.*

```
1 # -----
2 # Algorithme naïf de recherche d'un motif dans un texte
3 # -----
4
5
6 # Données de test :
7 # -----
8 le_ver = "Un ver vert va vers un verre vert à l'envers."
9
```

ALGORITHME NAÏF DE RECHERCHE

- Travail à faire :

- Implémenter dans votre fichier le code de la fonction `est_present()` :

```
def est_present(motif: str, texte: str) -> bool :  
    """Recherche la 1er occurrence du motif dans texte.  
    Retourne true si le motif est présent et false en cas d'absence."""  
  
    present = False  
  
    # Boucle principale : déplacement de la fenêtre de gauche à droite  
    i_dep = 0  
    i_fin = len(texte) - len(motif)  
  
    for i in range(i_dep, i_fin + 1, 1):  
        # Boucle secondaire : correspondance du motif sur le fragment de texte  
        for j in range (len(motif)):  
            if motif[j] != texte[i + j]:  
                break # Fin de la boucle secondaire dès qu'une lettre ne correspond pas  
            if j == len(motif) - 1:  
                present = True  
                return present  
  
    return present
```


ALGORITHME NAÏF DE RECHERCHE

- Travail à faire :

- Procédons aux tests de notre fonction avec `est_present()` le code suivant :

```
# Tests :  
# -----  
print("Tests de la fonction est_present() :\n"  
      "-----")  
mot_test = "vert"  
print(f'''Le mot "{mot_test}" est présent dans : "{le_ver}" : {est_present(mot_test, le_ver)}''')  
mot_test = "verte"  
print(f'''Le mot "{mot_test}" est présent dans : "{le_ver}" : {est_present(mot_test, le_ver)}''')
```

ALGORITHME NAÏF DE RECHERCHE

- **Travail à faire :**

- L'exécution de votre programme doit produire le résultat suivant :

```
>>> %Run 'Recherche naive.py'
Tests de la fonction est_present() :
-----
Le mot "vert" est présent dans : "Un ver vert va vers un verre vert à l'envers." : True
Le mot "verte" est présent dans : "Un ver vert va vers un verre vert à l'envers." : False
```

ALGORITHME NAÏF DE RECHERCHE

- Travail à faire :

- Le code proposé exécute un **break** dans la boucle secondaire de recherche du motif.
- Cette instruction est parfois critiquée, car elle permet généralement un code plus succinct, mais en même temps peut compliquer la lecture et le suivi du déroulement programme.

```
for i in range(i_dep, i_fin + 1, 1):  
    # Boucle secondaire : correspondance du motif sur le fragment de texte  
    for j in range(len(motif)):  
        if motif[j] != texte[i + j]:  
            break # Fin de la boucle secondaire dès qu'une lettre ne correspond pas  
        if j == len(motif) - 1:  
            present = True  
            return present
```

ALGORITHME NAÏF DE RECHERCHE

- Travail à faire :

- A la suite des tests, proposer une variante de la fonction `est_present()` que vous nommerez `est_present_sans_break()`
- Cette fonction remplacera le code de la seconde boucle `for` par une boucle `while`.
- Tester votre nouvelle fonction pour la valider.

```
def est_present_sans_break(motif: str, texte: str) -> bool :  
    """Recherche la 1er occurrence du motif dans text.  
    Retourne true si le motif est présent et false en cas d'absence."""  
  
    present = False  
  
    # Boucle principale : déplacement de la fenêtre de gauche à droite  
    i_dep = 0  
    i_fin = len(texte) - len(motif)  
  
    for i in range(i_dep, i_fin + 1, 1):  
        # Boucle secondaire : correspondance du motif sur le fragment de texte  
        nb_lettres_identiques = 0  
        j = 0  
        while A compléter...  
            ...  
            ...  
            ...  
            ...  
  
    return present
```

ALGORITHME NAÏF DE RECHERCHE

- Travail à faire :

- Avant d'aborder la recherche selon l'algorithme de **Boyer-Moore-Horspool**, nous vous proposer d'implémenter 2 fonctions complémentaires :

- Une fonction qui retourne le nombre d'occurrences du motif :

```
def nb_occurrences(motif: str, texte: str) -> int:  
    """Retourne le nombre d'occurrences du motif dans texte."""
```

- Une fonction qui retourne une liste contenant l'index de départ des différentes occurrences du motif :

```
def index_occurrences(motif: str, texte: str) -> [int]:  
    """Retourne une liste contenant l'index du 1er caractère des occurrences de motif dans texte."""  
    liste_index = []
```

ALGORITHME NAÏF DE RECHERCHE

- Travail à faire :

- Une fois les 2 fonctions implémentées, vous pourrez les tester avec le code suivant :

```
# Tests :  
# -----  
print("\n"  
      "Tests de la fonction nb_occurrences() :\n"  
      "-----")  
mot_test = "vert"  
print(f'''Le mot "{mot_test}" a {nb_occurrences(mot_test, le_ver)} occurrence(s) dans : "{le_ver}"""
```

- Résultat attendu :

```
Tests de la fonction nb_occurrences() :  
-----  
Le mot "vert" a 2 occurrence(s) dans : "Un ver vert va vers un verre vert à l'envers."
```

ALGORITHME NAÏF DE RECHERCHE

- **Travail à faire :**

- Une fois les 2 fonctions implémentées, vous pourrez les tester avec le code suivant :

```
# Tests :  
# -----  
print("\n"  
      "Tests de la fonction index_occurrences() :\n"  
      "-----")  
mot_test = "vert"  
print(f'''Le mot "{mot_test}" a pour liste d'occurrences : {index_occurrences(mot_test, le_ver)}\ndans : "{le_ver}''')
```

- **Résultat attendu :**

```
Tests de la fonction index_occurrences() :  
-----  
Le mot "vert" a pour liste d'occurrences : [7, 29]  
dans : "Un ver vert va vers un verre vert à l'envers."
```

L'ALGORITHME BOYER-MORRE-HORSPOOL

Présentation de l'algorithme

PRÉSENTATION DE L'ALGORITHME

- **Critique de l'algorithme naïf :**

- Notre algorithme naïf est opérationnel, mais il fait « bêtement » avances la fenêtre de contrôle du motif par incrément de 1.
- L'algorithme de **Boyer-Moore-Horspool** va mettre en place une stratégie qui va nous permettre de déplacer efficacement notre fenêtre de validation.
- En contre partie, un prétraitement sera réalisé pour préparer les futurs déplacements.
- Autre aspect de cet algorithme, il vérifiera la correspondance des lettres du motif en réalisant une lecture **de la droite vers la gauche** dans la fenêtre de contrôle.

PRÉSENTATION DE L'ALGORITHME



Principe général

- L'algorithme **compare le motif au texte de droite à gauche**, mais il saute des positions dans le texte au lieu de toujours glisser d'un seul caractère (comme le ferait un algo naïf).

PRÉSENTATION DE L'ALGORITHME

Fonctionnement :

1. Prétraitement (construction de la table de décalage) :

1. Pour chaque caractère du motif (sauf le dernier), on enregistre combien de caractères on peut "sauter" si ce caractère cause un échec lors de la comparaison.
2. Si un caractère ne figure pas dans le motif, on pourra sauter tout le motif.

2. Recherche dans le texte :

1. On aligne le motif avec le texte.
2. On compare de droite à gauche.
3. Si ça ne matche pas, on utilise la table de décalage pour déterminer combien de positions on peut décaler le motif vers la droite.

PRÉSENTATION DE L'ALGORITHME

Exemple :

Itération : 1

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44		
texte :	U	n		v	e	r			v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à		l	'	e	n	v	e	r	s	.	
motif :	v	e	r	t																																											

echec

Itération : 2

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44		
texte :	U	n		v	e	r		v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à		l	'	e	n	v	e	r	s	.		
motif :				v	e	r	t																																								

echec

Itération : 3

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44		
texte :	U	n		v	e	r		v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à		l	'	e	n	v	e	r	s	.		
motif :								v	e	r	t																																				

réussite

PRÉSENTATION DE L'ALGORITHME

Exemple :

Itération : 4

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	
texte :	U	n		v	e	r		v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à		l	'	e	n	v	e	r	s	.	
motif :												v	e	r	t																															

echec

Itération : 5

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	
texte :	U	n		v	e	r		v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à		l	'	e	n	v	e	r	s	.	
motif :																v	e	r	t																											

echec

Itération : 6

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	
texte :	U	n		v	e	r		v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à		l	'	e	n	v	e	r	s	.	
motif :																				v	e	r	t																							

echec

PRÉSENTATION DE L'ALGORITHME

Exemple :

Itération : 7

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44			
texte :	U	n			v	e	r			v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à			l	'	e	n	v	e	r	s	.
motif :																									v	e	r	r	t																			

echec

Itération : 8

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44			
texte :	U	n			v	e	r			v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à			l	'	e	n	v	e	r	s	.
motif :																										v	e	r	t																			

echec

Itération : 9

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44			
texte :	U	n			v	e	r			v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à			l	'	e	n	v	e	r	s	.
motif :																											v	e	r	t																		

echec

PRÉSENTATION DE L'ALGORITHME

Exemple :

Itération: 9

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	
texte :	U	n		v	e	r		v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e	v	e	r	t		à		l	'	e	n	v	e	r	s	.		
motif :																											v	e	r	t																

echec

Itération: 10

index :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	
texte :	U	n		v	e	r		v	e	r	t		v	a		v	e	r	s		u	n		v	e	r	r	e		v	e	r	t		à		l	'	e	n	v	e	r	s	.	
motif :																														v	e	r	t													

réussite

On poursuit jusqu'à la fin du texte...

L'ALGORITHME BOYER-MORRE- HORSPOOL

Implémentation de la fonction

IMPLÉMENTATION DE LA FONCTION

- Rédaction de l'entête :

- Faire la saisie de l'entête de la fonction

```
def est_present_bmh(motif: str, texte: str) -> bool :  
    """Recherche la 1er occurrence du motif dans texte.  
    Retourne true si le motif est présent et false en cas d'absence."""
```

- La 1ère étape consiste à renseigner le dictionnaire de décalage qu'on appellera **dico_deca**.

IMPLÉMENTATION DE LA FONCTION

- **Code pour le dictionnaire dico_deca :**

- Copier et compléter le code assurant l'initialisation de **dico_deca** dans le corps de votre fonction **est_present_bmh()**.

```
def est_present_bmh(motif: str, texte: str) -> bool :  
    """Recherche la 1er occurrence du motif dans texte.  
    Retourne true si le motif est présent et false en cas d'absence."""  
  
    # Construction du dictionnaire de décalage :  
    # A pour clé les lettres du motifs et pour valeur les  
    # position relatives des lettres du motif par rapport à la dernière.  
    # Remarque : Si une lettre apparait plusieurs fois, on appliquera la distance la plus courte  
    dico_deca = {}  
    i_fin_motif = len(motif) - 1  
    for i in range(...?????????): # On exclut la dernière lettre  
        dico_deca[motif[i]] = i_fin_motif - i
```

IMPLÉMENTATION DE LA FONCTION

- **Code pour le déplacement de la fenêtre et recherche du motif :**
 - Nous allons commencer par initialiser notre boucle **while** de recherche du motif dans le texte :

```
for i in range(i_fin_motif): # On exclut la dernière lettre
    dico_deca[motif[i]] = i_fin_motif - i

# Boucle de déplacement de la fenêtre de contrôle :
i_dep_droit = len(motif) - 1
i_fin_texte = len(texte) - 1
i = i_dep_droit
present = False

while i <= i_fin_texte and not present:
```

IMPLÉMENTATION DE LA FONCTION

- **Code pour le déplacement de la fenêtre et recherche du motif :**
 - Copier et compléter le code assurant l'identification du motif et le déplacement de la fenêtre de recherche du motif :

```
while i <= i_fin_texte and not present:
    # Boucle d'identification du motif dans la fenêtre de texte
    nb_lettres_identiques = 0
    j = len(motif) - 1
    while j >= 0 and motif[j] == texte[i - (len(motif) - 1 - j)]:
        nb_lettres_identiques += 1
        j -= 1
    # On vérifie si le motif a été trouvé :
    if nb_lettres_identiques == len(motif):
        present = True

    # Mise à jour de l'index de recherche i
    if texte[i] in dico_deca :
        i += dico_deca[ ... ?????? ]
    else:
        i += ... ??????

return present
```

IMPLÉMENTATION DE LA FONCTION

- **Code pour le déplacement de la fenêtre et recherche du motif :**
 - Procéder au test de votre fonction `est_present_bmh()`.
 - Si votre code est opérationnel, créer les 2 fonctions suivantes et procéder à leur test :
 - `nb_occurrences_bmh(motif: str, texte: str) -> int`
 - `index_occurrences_bmh(motif: str, texte: str) -> [int]`

Fin